SMOOTH · RESEARCH FOR MARCO · 3 NOV 2026

Could Smooth build your day instead of copying Canvas?

The plain version of the research. What already exists, what is really missing, what I would build, what it costs, and whether the AI we use now is up to it.

Your idea, in one line

Gather every page Smooth can see (modules, announcements, pages, syllabus, Ed) into one place, and let an AI build the day-to-day from all of it. That way an assignment that only lives in a module shows up, and an announcement that cancels one takes it away.

> **Short answer:** yes, and most of the collecting is already done. The thing to build is smaller and safer than "let the AI write the list": let the AI read each new post and report *what changed*, then let normal code apply that to the list you already have.

What Smooth already does (more than you'd think)

I had the code surveyed. Every hour Smooth already pulls, per class:

- the calendar feed (what the dashboard is built from today)
- every assignment with its description and whether you turned it in
- every module and the items inside it
- the first 40 pages, newest first
- the syllabus (page, body or PDF)
- announcements from the last 120 days
- Ed posts from staff

- Canvas's own "Planner" to-do list (but only to fill in missing dates)

And the dashboard already has cards from six places: the feed, Gradescope, module readings, tasks found in announcements and Ed posts, syllabus exams, and things you add yourself.

So what is actually missing?

Every one of those paths can only **add**. None of them can say "this post changes that card."

> *Professor posts: "HW3 is cancelled. The quiz moves from Wednesday to Friday."*
>
> `today` The announcement reader can only create a new task. The HW3 card from the feed stays. The quiz might get a second card with the new date, or get dropped as a duplicate of the old one. Nothing connects the post to the cards.
>
> `with this plan` HW3 is tagged "cancelled · Oct 3 announcement" and leaves your todo. The quiz card shows Friday with Wednesday struck out. Tap either to see the post, or to undo.

The second gap is that the course material we already pool on the server is never read by any AI except the syllabus summary. The chat ("the tree") gets a small hand-picked slice each time.

What I would build: "facts", not a rewritten list

The version in your idea, where the AI reads everything and writes the day, has four problems that each matter on their own:

- **Losing work.** Smooth's one rule is that nothing a student did is ever lost. A list a model regenerates nightly breaks that the first time it forgets an item.
- **Jitter.** The list would reorder or reword itself each night because the model sampled differently.

- **Cost.** Re-reading all of a student's material every night is the expensive design (numbers below).
- **Trust.** A cancelled card has to say which post cancelled it, and let you undo. That needs a source, not a rewrite.

So instead, the model reads **each new post once** and returns small structured "facts":

1. **A new post arrives**
   (announcement, Ed post, page edit, module text). We already know it is new, because we hash every document.
2. **The model reads it with the class's current item list beside it**
   (titles, ids, dates, about 30 short lines) and returns facts like *cancel item #4512*, *reschedule item #4520 to Fri Nov 7*, *new task "read ch. 7 before Tuesday"*. Each fact must include a quoted sentence from the post.
3. **Code checks the fact**
   : the quote really is in the post, the item id really exists, the date is sane. Anything that fails is thrown away. This is the guard against invented tasks.
4. **Your phone applies it to your list**
   with clear precedence: your own edits always win, then what Canvas itself says, then the fact. Nothing is deleted, cards get a tag and an undo, and the megaphone becomes "what changed".

One more design choice that matters: the facts are computed **once per class on the server**, where we already pool course material, and shared by everyone in that class. "HW3 is cancelled" means the same thing for every student. Only the application to *your* list, with your completions and edits, happens on your phone. That keeps the privacy line exactly where it is today.

After the facts are applied, "today" needs no AI at all. It is just: what is overdue or due in two days, plus prep tasks before tomorrow's lecture, plus your class meetings minus any cancelled ones.

Two reads, not a schedule

This does not run every night. It is two different reads with two different jobs.

Read 1: the first read, once per class

When a class first connects, the best model we can afford reads everything that class has in one sitting: the syllabus, every module with every item in order, the subheaders between them, every page, every assignment description. It writes a **semester map**, which is notes, not a task list:

- which module is which week, and which dates that means, using the class meeting pattern
- the rhythm of the class: "quiz every Friday", "prelab before each lab", "readings per week, never dated"
- every item it found, with the date Canvas states or the date it inferred from position, and which of the two it is
- standing rules, exam and milestone dates that live only in the syllabus or a schedule page
- what it could not pin down, so those get flagged instead of guessed

This is the expensive read, and it happens once per class, shared by every student in it. That makes it the place to spend on the premium tier: ten to fifty cents per class. It is re-read only when the modules or syllabus change. On your phone, items the map found that Canvas has not published yet show in "all" as `expected` cards with "from module 9" as the source, months out. When the real assignment appears, the expected card merges into it.

Read 2: each new post, read against the map

When the hourly sync finds a new or changed document, the model reads it with the semester map and the class's current item list beside it. "The usual quiz is pushed to Monday" is only readable by something that already knows there is a usual quiz. It returns facts as above, can amend the map ("week 9 reading replaced"), and can say "needs a look" when a post is ambiguous. Event-driven, cheap, no schedule.

Edge cases this handles that today's app does not

REMOVING OR CHANGING WORK	BETTER ASSIGNMENTS FROM POSTS	READING MODULES AND PAGES PROPERLY

- "HW3 is cancelled"
- made optional
- unpublished or deleted in Canvas (today the card lingers up to 14 days)
- "no class Friday"
- exam moved or cancelled
- conditionals ("skip HW5 if...") flagged, not applied
- due date moved, for all or one section
- scope narrowed ("only problems 1-4")
- "submit on Gradescope, not Canvas"
- a date for an undated item
- "quiz covers ch. 5-6" as a note
- prep tasks ("read 3.1-3.3 before Tuesday")
- "in two weeks" resolved from the post date
- corrections superseding earlier posts
- the professor's own plans kept out
- work that only exists as a module item, page or file
- undated weekly readings dated by week and lecture day
- undated items bracketed by dated neighbours
- "quiz every Friday" stated once becomes the whole term
- a schedule table with dates months out
- project milestones on a project page
- a schedule page edited mid-term
- last year's dates on a recycled site, flagged

Across sources: the same work in the feed, a module and Gradescope shown as one card; a syllabus date and a later announcement date disagreeing, newer wins with the source shown. **Still out of reach:** anything said only in lecture, email or Slack; private Ed threads; a dead Canvas session.

What has to change for this to work

Grouped by layer. The collection items touch the sync path Olisa owns and need his go first.

One identity per piece of work (the dedup engine)

- Today each source has its own identity and duplicates are caught by comparing titles. Instead, every work item gets a set of aliases: the Canvas assignment id, the module item id, the Gradescope id, the map's id, the post that created it. Exact ids merge first, title-and-date closeness second, the model's pick third and only among listed candidates.

- Merges are recorded and reversible ("these are two things" splits them). Two different Canvas ids never collapse, which keeps the Lab 3 / Lab 3 Report lesson.
- Facts and expected cards are stored like the ledger: nothing is ever deleted.

Announcements become part of the card

- A fact links the post to the item it changes, so a card carries its history ("moved to Fri · Oct 3 post · scope narrowed · Oct 5 post") and opens the post. The separate announcement-task list goes away.
- The megaphone shows "what changed", grouped by item, plus a "needs a look" group. Raw posts only when they touched nothing.
- Cards get a tag, a struck-out old date, an "expected" style, undo, and one-tap source.

Collection (Olisa's area)

- Fetch what Canvas states as data and we skip today: per-section due dates, module completion rules and unlock dates, Canvas's Planner as a real source, missing submissions, discussions, page to-do dates, every page instead of the first 40, schedule PDFs.
- One document per module item instead of one per module, so changes are caught at item level.
- Detect deleted assignments by comparing successive API lists, not just the feed.
- Respect Canvas's rate-limit header. Stop uploading "Status: submitted" in assignment text.

Server

- A semester-map table and a build step triggered when a class's modules or syllabus change, with a per-class budget.
- A fact-extraction function replacing the announcement reader, run when new documents arrive; facts stored per document and served to phones.
- Strict output schemas enforced by the provider (today the server only asks for "some JSON"), plus the quote check.
- Quotas per class and a global monthly cap, never charged to a student's chat budget.
- Move the chat off gpt-4.1-mini before January; pick the extraction model from the test set; use the premium tier for the first read only.

Making it reason well

- Everything the model needs is in front of it: the map, the item list, the meeting pattern, the post's date and weekday. Code does all date arithmetic and all precedence.
- Newer facts supersede older ones on the same item. Corrections fall out of the same rule.
- "Needs a look" is a real output with a real screen, so the model is rewarded for flagging instead of guessing.
- The test set grows a dedup section: pairs that must merge and pairs that must not.

What it costs

The model cost is small either way. The design matters far more than the model.

≈ $2 / mo

ongoing: new posts read once per class, 200 students, a $0.10-per-million model

$15 – 75 once

the first read of 150 classes on a premium model; a couple of dollars on the cheap tier

$50 – 1,000 / mo

for comparison: re-reading everything per student nightly, 200 students

Why the gap: the ongoing cost scales with how many posts a class makes (a few a week), not how many students use the app, and the big read happens once per class. Prompt caches would not rescue a nightly design, because every provider's cache expires within an hour.

The real cost is engineering, roughly five pieces plus a test set:

PIECE	WHERE	WHAT
Structured sources	phone	Read the things Canvas already states as data: Planner items, module completion rules, missing submissions, per-section due dates, page to-do dates. No AI. Catches "it's only in the modules" on its own.

Fact extraction	server	A new function that reads each new post once and stores facts per class. Strict output format, quote check.
Applying facts	phone	The precedence rules, tags on cards, undo, "what changed" in the megaphone.
The day view	phone	Today = due soon + prep tasks + meetings, computed, not generated.
Feedback	both	A thumbs-down on a wrong fact, so accuracy can be tracked per model.

Canvas side: no money. About six extra requests per class per hourly sync, all with the normal login, no developer key. Canvas has no "what changed" feed for students, so change detection stays our own diff, which already exists. Everything inherits the sign-in fragility you know about: if the session is dead, facts just go stale.

Is the AI we use now good enough?

Three findings, two of which matter whether or not we do this.

1. **The chat's model is being retired.** "The tree" runs on OpenAI's gpt-4.1-mini. OpenAI's own deprecation page says it shuts down on **14 Jan 2027**, replacement gpt-6-luna. I read the page myself. It has to move within two months regardless.
2. **The announcement reader runs on the weakest-evidence model.** It uses glm-5.3-flash, the cheapest option by three times. Third-party quality scores for it disagree with each other, its response time is the most erratic (which is why the chat has a backup model race), and it is the "always-thinking" model that once used its whole output budget on thinking and returned nothing. Fine as a backup. Not what I'd trust to decide a homework is cancelled.
3. **No public benchmark measures this exact job** (messy post in, correct dated changes out, nothing invented). Proxies exist, but a model good at trivia can still invent a task.

CANDIDATE	PRICE IN / OUT PER 1M TOKENS	READ
Claude Haiku 5.5	`$0.10 / $0.50`	Same price as gpt-6-luna, scores higher on general ability, and enforces the output format at the decoder. Front-runner.

gpt-6-luna	`$0.10 / $0.50`	OpenAI's named successor to what the chat uses now. If it tests well, one model could serve both. Lowest general-ability score of the group, so test first.
DeepSeek V4.1 Flash	`$0.045 / $0.50`	Cheap dark horse with the best "doesn't make things up" score on the one leaderboard that measures it.
gpt-6.1-terra	`$1.00 / $5.00`	The quality fallback if nothing cheap is accurate enough. Still only about $18 a month at 200 students in the pooled design.
glm-5.3-flash (today)	`$0.03 / $0.09`	Keep as the backup only.

My recommendation: don't pick from leaderboards. Take 50 to 80 real posts off your phone and Olisa's, write down by hand what each one should produce (including the trick ones, like a professor describing his own plans), run the four candidates against that, and pick on accuracy first. A missed cancellation is what you have today. A wrong one hides real work, so that number has to be near 100%.

The plan, in order

Each step ships on its own and is useful without the next.

1. **The test set and the model migration.** Build the labelled posts, write the prompt, bench four models. Separately, move the chat off gpt-4.1-mini before January. No app change yet.
2. **Structured Canvas sources, no AI.** Works for every school, not just Penn. Covers the "only in modules" case by itself.
3. **Fact extraction on the server**, pooled per class, with its own quota so it never eats anyone's chat budget.
4. **Applying facts on the phone**, with tags, undo, and the megaphone as "what changed".
5. **The day view.**
6. **Thumbs-down feedback** to track accuracy over time.

Four things you and Olisa need to decide first

1. **Pooled on the server, or per phone?** I say pooled: 50 to 200 times cheaper at scale, and every student gets the same reading of the same post. Per phone is how the announcement reader works today.
2. **Hide or tag?** When a fact says "cancelled", does the card leave your todo, or just get a tag until you confirm? Hiding is what you asked for. Tagging is safer for the first build. I'd hide with undo, once the test set shows the accuracy.
3. **Which model**, after the test, and whether the chat moves to the same one.
4. **Who owns it.** Olisa has owned announcements and Ed since mid-October, and this replaces what the announcement reader produces. It is his call as much as yours.

The full write-up with code references, API details and sources is `docs/DAY_PLANNER_RESEARCH_2026-11-03.md` on the v8 branch. Prices read 3 Nov 2026 from OpenRouter, OpenAI and Anthropic. Nothing here is built or agreed.